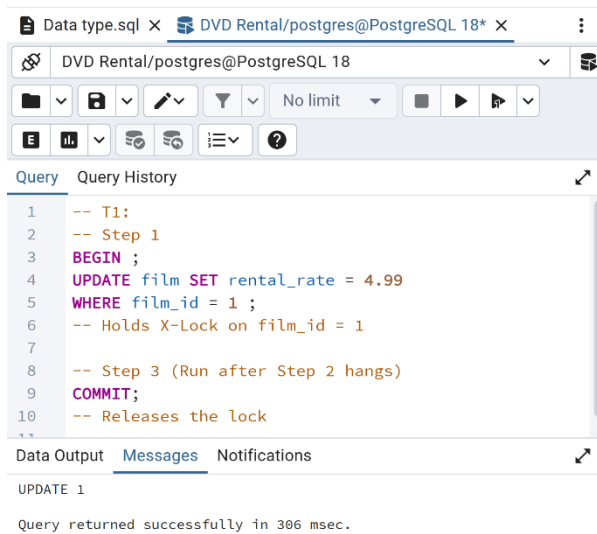


Lab 1:

- Run step 1:

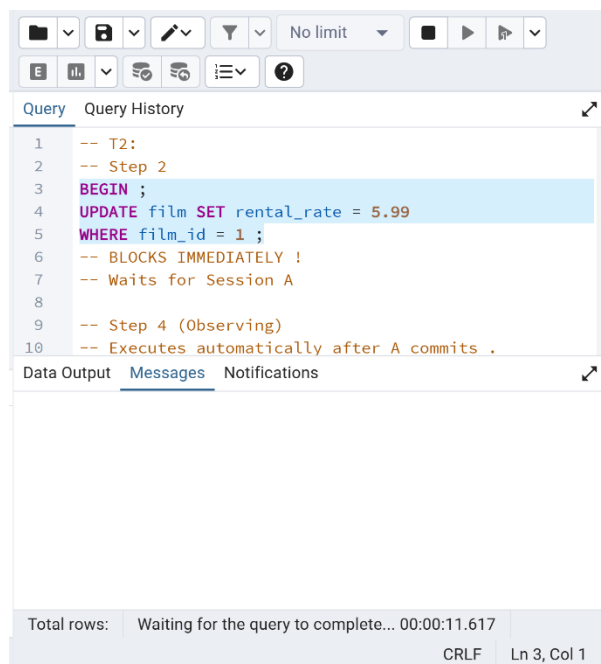


```
1  -- T1:
2  -- Step 1
3  BEGIN ;
4  UPDATE film SET rental_rate = 4.99
5  WHERE film_id = 1 ;
6  -- Holds X-Lock on film_id = 1
7
8  -- Step 3 (Run after Step 2 hangs)
9  COMMIT;
10 -- Releases the lock
```

UPDATE 1

Query returned successfully in 306 msec.

- Run step 2:



```
1  -- T2:
2  -- Step 2
3  BEGIN ;
4  UPDATE film SET rental_rate = 5.99
5  WHERE film_id = 1 ;
6  -- BLOCKS IMMEDIATELY !
7  -- Waits for Session A
8
9  -- Step 4 (Observing)
10 -- Executes automatically after A commits .
```

Total rows: Waiting for the query to complete... 00:00:11.617

CRLF Ln 3, Col 1

- Run step 3:

Query

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.99!
11 COMMIT;

```

Data Output

rental_rate	numeric (4,2)
1	4.99

Total rows: 1 Query complete 00:00:00.768 CRLF Ln 2, Col 1

- Run step 2:

Query

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.09!
11 COMMIT;

```

Data Output

rental_rate	numeric (4,2)
1	4.99

Total rows: 1 Query complete 00:00:00.768 CRLF Ln 10, Col 20

Query

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 0.99
4  WHERE film_id = 10;
5
6  COMMIT;
7  -- Modifies data and commits
8  -- while A is still active .
9

```

Data Output

COMMIT

Query returned successfully in 160 msec.

Total rows: Query complete 00:00:00.160 CRLF Ln 5, Col 1

- Run step 3:

Query History

```

1  -- Step 1
2  BEGIN;
3  SELECT rental_rate FROM film
4  WHERE film_id = 10;
5  -- Result : 4.99 (S-Lock released)
6
7  -- Step 3
8  SELECT rental_rate FROM film
9  WHERE film_id = 10;
10 -- NEW RESULT : 0.09!
11 COMMIT;

```

Data Output Messages Notifications

Showing rows: 1 to 1

	rental_rate numeric(4,2)
1	0.99

Total rows: 1 Query complete 00:00:00.323 CRLF Ln 8, Col 1

Lab 3:

- Run step 1:

Query History

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 100;
5
6  -- Step 3
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 200;
9  -- Blocks ( Waiting for B)

```

Data Output Messages Notifications

UPDATE 1

Query returned successfully in 172 msec.

Total rows: 1 Query complete 00:00:00.172 CRLF Ln 2, Col 1

- Run step 2:

DVD Rental/postgres@PostgreSQL 18

Query

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 100;
5
6  -- Step 3
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 200;
9  -- Blocks ( Waiting for B)

```

Data Output

UPDATE 1

Query returned successfully in 28 secs 658 msec.

Total rows: Query complete 00:00:28.751 CRLF Ln 7, Col 1

DVD Rental/postgres@PostgreSQL 18

Query

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 2.99
4  WHERE film_id = 200;
5
6  -- Step 4
7  UPDATE film SET rental_rate = 3.99
8  WHERE film_id = 100;
9  -- BOOM! ERROR : deadlock detected

```

Data Output

ERROR: deadlock detected
Process 14908 waits for ShareLock on transaction 987; blocked by process 16596.
Process 16596 waits for ShareLock on transaction 988; blocked by process 14908.

SQL state: 40P01
Detail: Process 14908 waits for ShareLock on transaction 987; blocked by process 16596.
Process 16596 waits for ShareLock on transaction 988; blocked by process 14908.
Hint: See server log for query details.
Context: while locking tuple (54.2) in relation "film"

Total rows: Query complete 00:00:01.286 CRLF Ln 7, Col 1

Lab 4:

- Run step 1:

Query

```

1  -- Step 1
2  BEGIN ;
3  SELECT * FROM inventory
4  WHERE inventory_id = 10 FOR UPDATE;
5  -- Locks the row proactively
6
7  -- Step 3
8  INSERT INTO rental
9  (rental_date, inventory_id, customer_id, staff_id)
10 VALUES (NOW( ), 10, 1, 1);
11 COMMIT;

```

Data Output

Showing rows: 1 to 1

Page 1 of 1

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2006-02-15 10:09:17

Total rows: 1 Query complete 00:00:00.293 CRLF Ln 2, Col 1

- Run step 2:

The screenshot displays the MySQL Workbench environment with two instances of the Query Editor and their respective output windows.

Left Panel - Query Editor

```
1 -- Step 1
2 BEGIN ;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- Locks the row proactively
6
7 -- Step 3
8 INSERT INTO rental
9 (rental_date, inventory_id, customer_id, staff_id)
10 VALUES (NOW( ), 10, 1, 1);
11 COMMIT;
```

Right Panel - Query Editor

```
1 -- Step 2
2 BEGIN;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- BLOCKS IMMEDIATELY
6
7 -- Step 4
8 -- Unblocks after A commits .
9 -- App logic detects it's already rented!
10 ROLLBACK;
```

Data Output (Left)

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

Status Bar (Left)

Total rows: 1 Query complete 00:00:00.293 CRLF Ln 2, Col 1

Messages (Right)

Total rows: Waiting for the query to complete... 00:00:09.808 CRLF Ln 2, Col 1

- Run step 3:

The screenshot displays the DBeaver SQL interface with two queries being compared:

- Left Query (Query History):**

```

-- Step 1
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- Locks the row proactively

-- Step 3
INSERT INTO rental
(rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;
```
- Right Query (Query History):**

```

-- Step 2
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- BLOCKS IMMEDIATELY

-- Step 4
-- Unblocks after A commits .
-- App logic detects it's already rented!
ROLLBACK;
```

Below the queries, the "Data Output" tab shows the results of the first query:

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

Status bars at the bottom indicate successful completion: "Query returned successfully in 232 msec." for the left and "Query complete 00:00:04.567" for the right.

- Run step 4:

Query	Query History
<pre> 1 -- Step 1 2 BEGIN; 3 SELECT * FROM inventory 4 WHERE inventory_id = 10 FOR UPDATE; 5 -- Locks the row proactively 6 7 -- Step 3 8 INSERT INTO rental 9 (rental_date, inventory_id, customer_id, staff_id) 10 VALUES (NOW(), 10, 1, 1); 11 COMMIT;</pre>	<pre> 1 -- Step 2 2 BEGIN; 3 SELECT * FROM inventory 4 WHERE inventory_id = 10 FOR UPDATE; 5 -- BLOCKS IMMEDIATELY 6 7 -- Step 4 8 -- Unblocks after A commits . 9 -- App logic detects it's already rented! 10 ROLLBACK;</pre>
Data Output Messages Notifications	Data Output Messages Notifications
COMMIT	ROLLBACK
Query returned successfully in 232 msec.	Query returned successfully in 152 msec.